

Analisis Protokol Lapisan Transport: UDP, TCP, dan SCTP

Kahlil Gibran

Program Studi Informatika

NIM: 064002100002

Abstrak- Lapisan transport berperan sebagai penghubung antara aplikasi dan jaringan IP dengan menyediakan komunikasi proses-ke-proses. Tulisan ini membahas User Datagram Protocol (UDP), Transmission Control Protocol (TCP), dan Stream Control Transmission Protocol (SCTP). UDP memberikan layanan datagram sederhana dengan overhead kecil, TCP menyediakan aliran byte andal melalui koneksi, sedangkan SCTP menggabungkan orientasi pesan, keandalan, multistreaming, dan multihoming. Pembahasan meliputi definisi, alasan kebutuhan, kegunaan, cara kerja, implementasi, serta perbandingan karakteristik. Hasil kajian menunjukkan bahwa tidak ada satu protokol yang selalu unggul; pemilihan protokol bergantung pada kebutuhan aplikasi terhadap latensi, reliabilitas, urutan data, toleransi kegagalan, dan dukungan infrastruktur.

Kata kunci- UDP, TCP, SCTP, lapisan transport, reliabilitas, jaringan komputer.

I. PENDAHULUAN

Komunikasi data di Internet tidak hanya memerlukan alamat jaringan, tetapi juga mekanisme agar data dari suatu aplikasi sampai ke aplikasi tujuan yang benar. Lapisan network, seperti IP, mengirimkan paket antarhost, sedangkan lapisan transport mengatur komunikasi antarproses melalui nomor port dan aturan pertukaran data. Karena aplikasi memiliki kebutuhan berbeda, lapisan transport menyediakan beberapa pilihan protokol.

UDP, TCP, dan SCTP merupakan contoh penting protokol transport. UDP bersifat sederhana dan tidak membangun koneksi. TCP membangun koneksi dan memastikan data diterima secara andal serta berurutan. SCTP dirancang untuk menyediakan komunikasi andal berbasis pesan dengan kemampuan tambahan seperti multistreaming dan multihoming.

Tulisan ini bertujuan menjelaskan definisi, kebutuhan, kegunaan, cara kerja, dan implementasi UDP, TCP, dan SCTP. Selain itu, tulisan ini membandingkan ketiganya agar pemilihan protokol dapat disesuaikan dengan karakter aplikasi.

II. KONSEP LAPISAN TRANSPORT

Lapisan transport berada di atas lapisan Internet dan di bawah lapisan aplikasi. Tugas utamanya adalah multiplexing dan demultiplexing data aplikasi, segmentasi data, pengendalian kesalahan, pengaturan urutan, pengendalian aliran, dan dalam beberapa protokol pengendalian kemacetan. Tidak semua fungsi tersebut selalu disediakan oleh setiap protokol. UDP hanya memberikan layanan minimum, sementara TCP dan SCTP menyediakan fungsi keandalan yang lebih lengkap.

Konsep penting pada lapisan transport adalah port. Port memungkinkan satu host menjalankan banyak layanan sekaligus, misalnya web pada port 80 atau 443, DNS pada port 53, dan SMTP pada port 25. Dengan kombinasi alamat IP dan nomor port, sistem operasi dapat menyerahkan data kepada proses aplikasi yang tepat.

Perbedaan desain protokol transport biasanya muncul dari trade-off antara kecepatan, overhead, reliabilitas, dan kompleksitas. Aplikasi real-time dapat memilih latensi rendah meskipun sebagian paket hilang, sedangkan transfer file dan web membutuhkan jaminan data utuh dan terurut.

III. USER DATAGRAM PROTOCOL (UDP)

UDP didefinisikan dalam RFC 768 sebagai protokol datagram sederhana yang menyediakan pengiriman pesan antarproses melalui port [1]. Header UDP hanya terdiri dari source port, destination port, length, dan checksum. Desain ini membuat UDP ringan dan cepat karena tidak perlu membangun koneksi, menyimpan status sesi, atau melakukan retransmisi otomatis.

UDP dibutuhkan ketika aplikasi lebih mengutamakan latensi rendah, kontrol di tingkat aplikasi, atau pola komunikasi request-response yang singkat. Contoh umum adalah DNS, DHCP, VoIP, game online, streaming tertentu, dan protokol modern yang membangun keandalan sendiri di atas UDP. Karena UDP tidak menjamin paket sampai, tidak menjamin urutan, dan tidak menghindari duplikasi, aplikasi harus menangani sendiri kebutuhan tersebut bila diperlukan.

Cara kerja UDP relatif sederhana. Aplikasi mengirimkan data ke socket UDP dengan alamat IP tujuan dan port tujuan. Sistem operasi menambahkan header UDP, kemudian datagram dibawa oleh IP. Di sisi penerima, datagram diserahkan ke aplikasi berdasarkan port tujuan. Apabila datagram hilang, terlambat, atau datang tidak berurutan, UDP tidak melakukan koreksi otomatis.

Dalam implementasi, kesederhanaan UDP memberi keuntungan pada aplikasi yang membutuhkan respons cepat. Namun, RFC 8085 menekankan bahwa aplikasi berbasis UDP tetap perlu memperhatikan pengendalian laju, ukuran datagram, NAT traversal, dan keamanan agar tidak memperburuk kemacetan jaringan [5].

IV. TRANSMISSION CONTROL PROTOCOL (TCP)

TCP adalah protokol transport berorientasi koneksi yang memberikan layanan reliable byte stream. Spesifikasi TCP modern dikonsolidasikan dalam RFC 9293 [2]. Berbeda dengan UDP yang mempertahankan batas pesan, TCP memperlakukan data aplikasi sebagai aliran byte. Aplikasi tidak perlu mengurus segmentasi, retransmisi, atau pengurutan data karena fungsi tersebut ditangani oleh TCP.

TCP dibutuhkan oleh aplikasi yang memerlukan keandalan tinggi, misalnya HTTP/HTTPS, FTP, SSH, SMTP, POP3, dan IMAP. Dalam aplikasi tersebut, kehilangan sebagian data dapat merusak dokumen, halaman web, arsip, atau sesi login. Karena itu TCP menyediakan nomor urut, acknowledgment, retransmisi, checksum, pengendalian aliran, dan pengendalian kemacetan.

Koneksi TCP diawali dengan three-way handshake: klien mengirim SYN, server membalas SYN-ACK, dan klien mengirim ACK. Setelah koneksi terbentuk, data dikirim dalam segmen yang memiliki sequence number. Penerima mengirim acknowledgment untuk menandai byte yang telah diterima. Apabila segmen tidak diakui dalam waktu tertentu, pengirim melakukan retransmisi. Koneksi ditutup melalui pertukaran FIN atau dapat dihentikan dengan RST dalam kondisi tertentu.

Pengendalian aliran TCP dilakukan melalui receive window, yaitu informasi kapasitas buffer penerima. Pengendalian kemacetan mencegah pengirim membanjiri jaringan [6]. Kelebihan TCP adalah reliabilitas dan kompatibilitas luas; kelemahannya adalah overhead koneksi serta kemungkinan head-of-line blocking ketika satu segmen hilang menahan data berikutnya.

V. STREAM CONTROL TRANSMISSION PROTOCOL (SCTP)

SCTP distandardisasi dalam RFC 9260 sebagai protokol transport yang reliable, message-oriented, dan mendukung beberapa stream dalam satu asosiasi [3]. Istilah asosiasi digunakan karena SCTP dapat melibatkan lebih dari satu alamat IP pada endpoint. SCTP awalnya banyak dikaitkan dengan signaling telekomunikasi, tetapi konsepnya relevan untuk aplikasi yang membutuhkan pesan andal, pemisahan stream, dan ketahanan jalur.

SCTP dibutuhkan untuk mengatasi beberapa keterbatasan TCP dan UDP. TCP andal tetapi berbasis byte stream dan dapat mengalami head-of-line blocking antarjenis data. UDP mempertahankan batas pesan tetapi tidak andal. SCTP menggabungkan orientasi pesan dengan keandalan, serta menyediakan multistreaming sehingga kehilangan pada satu stream tidak harus menahan stream lain dalam asosiasi yang sama.

Pembentukan asosiasi SCTP menggunakan four-way handshake dengan mekanisme cookie. Proses ini membantu mengurangi risiko serangan resource exhaustion karena server tidak langsung menyimpan seluruh status sebelum cookie dikembalikan oleh klien. Setelah asosiasi terbentuk, SCTP mengirim data dalam chunk. Satu paket SCTP dapat berisi beberapa chunk, baik chunk kontrol maupun chunk data.

Fitur penting SCTP adalah multihoming. Endpoint dapat memiliki beberapa alamat IP sehingga komunikasi dapat berpindah ke jalur alternatif ketika jalur utama gagal. SCTP juga mendukung delivery berurutan maupun tidak berurutan sesuai kebutuhan aplikasi. Namun, adopsi SCTP di Internet publik tidak seluas TCP dan UDP karena dukungan perangkat jaringan, firewall, NAT, dan API aplikasi tidak selalu tersedia.

VI. PERBANDINGAN UDP, TCP, DAN SCTP

Aspek	UDP	TCP	SCTP
Model data	Datagram/pesan	Aliran byte	Pesan dan stream
Koneksi	Tanpa koneksi	Berorientasi koneksi	Asosiasi
Keandalan	Tidak ada jaminan bawaan	Andal, terurut	Andal, multi-stream
Overhead	Rendah	Sedang-tinggi	Lebih kompleks
Contoh	DNS, VoIP, game	Web, SSH, email	SIGTRAN, multi-stream

Kelemahan	Aplikasi menangani kehilangan data	Head-of-line blocking	Dukungan lebih terbatas
-----------	------------------------------------	-----------------------	-------------------------

VII. IMPLEMENTASI DAN CONTOH PENGGUNAAN

Dalam praktik pemrograman jaringan, ketiga protokol biasanya digunakan melalui socket API. Pada UDP, aplikasi membuat socket datagram lalu mengirim pesan dengan alamat tujuan. Model ini cocok untuk transaksi singkat seperti query DNS. Pada TCP, aplikasi server melakukan bind, listen, dan accept, sedangkan klien melakukan connect. Setelah koneksi terbentuk, komunikasi dilakukan seperti membaca dan menulis stream.

Implementasi SCTP membutuhkan dukungan sistem operasi dan pustaka yang sesuai. Pada Linux, SCTP dapat digunakan melalui dukungan kernel dan socket API tertentu. Aplikasi dapat memanfaatkan beberapa stream untuk memisahkan jenis pesan, misalnya pesan kontrol dan pesan data, sehingga kehilangan satu pesan tidak selalu menghambat seluruh aliran komunikasi.

Pemilihan protokol harus mempertimbangkan konteks. Jika aplikasi mengirim data kecil dan respons cepat, UDP dapat lebih efisien. Jika aplikasi mengirim data yang harus utuh dan berurutan, TCP biasanya menjadi pilihan utama. Jika aplikasi membutuhkan pesan andal dengan beberapa stream dan toleransi kegagalan alamat, SCTP menjadi alternatif yang kuat meskipun dukungan ekosistemnya perlu diperiksa.

VIII. ASPEK KEAMANAN

Keamanan lapisan transport tidak hanya bergantung pada protokol dasar. UDP, TCP, dan SCTP memiliki checksum untuk mendeteksi kesalahan, tetapi checksum bukan mekanisme keamanan kriptografis. Kerahasiaan, integritas kuat, dan autentikasi biasanya disediakan oleh lapisan tambahan seperti TLS, DTLS, IPsec, atau mekanisme keamanan aplikasi.

UDP rentan disalahgunakan untuk refleksi dan amplifikasi apabila layanan merespons paket kecil dengan balasan besar tanpa validasi memadai. TCP memiliki risiko seperti SYN flooding, meskipun teknik mitigasi seperti SYN cookies dapat digunakan. SCTP menggunakan cookie pada proses asosiasi untuk mengurangi konsumsi sumber daya sebelum klien terbukti dapat menerima balasan dari server.

Praktik keamanan yang baik meliputi pembatasan laju, validasi input, penggunaan enkripsi ketika data sensitif dikirim, konfigurasi firewall yang tepat, dan pemantauan anomali trafik. Dengan demikian, pemilihan protokol harus disertai desain keamanan yang sesuai.

IX. KESIMPULAN

UDP, TCP, dan SCTP sama-sama berada pada lapisan transport, tetapi dirancang untuk kebutuhan yang berbeda. UDP unggul dalam kesederhanaan dan latensi rendah, TCP unggul dalam keandalan dan dukungan luas, sedangkan SCTP menawarkan kombinasi message-oriented reliability, multistreaming, dan multihoming. Perbedaan tersebut menunjukkan bahwa pemilihan protokol tidak boleh hanya berdasarkan popularitas, tetapi harus mengikuti kebutuhan

aplikasi.

Untuk aplikasi yang toleran terhadap kehilangan paket atau mengatur keandalan sendiri, UDP merupakan pilihan efisien. Untuk aplikasi yang membutuhkan data utuh dan kompatibilitas maksimal, TCP tetap menjadi protokol dominan. Untuk aplikasi yang membutuhkan pemisahan stream dan toleransi kegagalan jalur, SCTP dapat memberikan manfaat teknis yang tidak tersedia secara langsung pada TCP atau UDP.

REFERENSI

- [1] J. Postel, "User Datagram Protocol," RFC 768, Aug. 1980. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc768>
- [2] W. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, Aug. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9293.html>
- [3] R. Stewart, M. Tuexen, and K. Nielsen, "Stream Control Transmission Protocol," RFC 9260, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9260.html>
- [4] R. Braden, Ed., "Requirements for Internet Hosts - Communication Layers," RFC 1122, Oct. 1989. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1122>
- [5] L. Eggert, G. Fairhurst, and G. Shepherd, "UDP Usage Guidelines," RFC 8085, Mar. 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8085>
- [6] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," RFC 5681, Sep. 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5681>